

Conway's Game of Life Dev Log

Conway's Game of Life — Dev Log (Pre-Research / Post-Research)

Step 1 & 2: Screen + Grid Creation

Pre-Research (what I thought it would be): A simple 2D array for the grid — leaning toward booleans (true = live, false = dead/empty), though open to a better structure once seen in the tutorial. A main loop initializes the window/screen.

Post-Research (what it was): Confirmed correct. Main program loop initializes the window with a target FPS (set to 12 initially, to be increased later). A separate `Grid` class holds rows, columns, and a 2D boolean array. The boolean array represents the grid — true/false values determine each cell's color. Each cell is drawn as a rectangle slightly smaller than the designated cell size, creating visible spacing between cells. Initial test configuration: 15px cell size at 1080×1080 resolution (temporary, subject to change).

Step 3: Counting Live Neighbors

Pre-Research (what I thought it would be): For each cell, calculate its neighbor count by checking up, down, left, right. Unsure whether diagonals count as neighbors, or only the four orthogonal directions.

Post-Research (what it was): Diagonals are included. The neighbor check covers a full 3×3 grid around the main cell (all 8 surrounding cells, not just the 4 orthogonal ones).

Cell Scanning / Iteration Order

Pre-Research (what I thought it would be): A loop walking through the array checking each cell sequentially, using index offsets. Concern: checking every cell in the same fixed order every pass could introduce directional bias. Considered whether a randomized order, or a checkerboard-style pattern (as used in some simulations for a more natural feel), might be used instead.

Post-Research (what it was): Standard sequential order was used — top to bottom, left to right (0 to length, 0 to height). No randomization or checkerboard pattern was implemented.

Neighbor-Offset Implementation

Pre-Research (what I thought it would be): Unsure whether neighbor-checking would be hardcoded direction-by-direction, or handled more efficiently.

Post-Research (what it was): Implemented using a list of offset vectors (-1/+1 on x and y) rather than hardcoding each direction individually. The list of vectors is iterated over in a loop/snippet to check all neighbors.

Step 4: Implementing Simulation Rules

Pre-Research (what I thought it would be): Count how many neighbors are true, incrementing a counter for each live neighbor, then apply the rules based on that count.

Post-Research (what it was): Confirmed as predicted. For each cell, tally live neighbors, then apply Conway's standard rules: - A live cell with two or three live neighbors survives. - A live cell with more than three live neighbors dies. - A dead cell with exactly three live neighbors becomes alive.

Step 5: Starting / Stopping the Simulation

Pre-Research (what I thought it would be): Spacebar to pause, Escape key to close.

Post-Research (what it was): Matched the plan — not revisited in detail, but implemented as described.

Step 6: Initial Grid State

Pre-Research (what I thought it would be): Randomize the entire grid at initialization, then let the simulation run from there.

Post-Research (what it was): Matched the plan — implemented as described.

Overall Outcome

- Most pre-research predictions turned out correct — fewer unresolved questions than expected going in.
- The rest of the implementation was standard boilerplate (mouse input handling, rendering, etc.).

Known Open Issue (Unresolved)

- **Mouse-click bug:** Mouse-click detection is frame-rate dependent. Holding the mouse down causes cell toggling to flicker on/off inconsistently, due to how the “mouse is down” state is being checked each frame. Identified as a simple fix, but not yet applied as of this log.